

以時速 100 英里的速度打造值得信賴的 AI

來源：<https://codelabs.developers.google.com/codelabs/trustable-at-100-mph?hl=zh-tw>

步驟數：12

本程式碼研究室會逐步說明如何建構賽車教練模擬器，結合即時遙測、LLM 推理和編碼的人工指導，展示如何建立更值得信賴的 AI 系統。

目錄

1. [總覽](#)
2. [所需條件](#)
3. [為什麼可信賴的 AI 很重要](#)
4. [瞭解高速 AI 和模組化可信架構](#)
5. [建構遙測串流伺服器](#)
6. [建構賽車模擬器](#)
7. [準備 AI 推理的遙測資料](#)
8. [新增防護措施和編碼的人類專業知識](#)
9. [設計教練角色和使用者體驗](#)
10. [查看端對端架構](#)
11. [挑戰](#)
12. [總結與後續步驟](#)

1. 總覽

人工智慧已成為許多軟體系統的一部分，但建構 AI 應用程式與建構使用者信任的應用程式不同。在許多現實環境中，挑戰不只是生成回覆，如何生成及時、有根據、可採取行動且符合人類專業知識的回覆，是一大挑戰。

在本程式碼研究室中，您將建構賽車教練模擬器，以具體有趣的方式展示這些概念。應用程式會使用虛擬賽車的遙測資料，在賽道上呈現動畫，並產生教練指導。雖然這是賽車情境，但同樣的架構概念也適用於醫療保健、製造、物流和其他重視信任的領域。

您將處理高速傳輸的遙測資料串流，將資料轉換成 AI 推論可有效使用的形式，並結合以 LLM 為基礎的輸出內容和編碼的人工指引，生成更值得信賴的回覆。

建構目標

在本程式碼研究室中，您將建構可信賴的 AI 原型，該原型具有下列功能：

- 從 Google Cloud 中執行的虛擬賽車串流遙測資料
- 使用 Chrome 顯示車輛在賽道上移動的視覺化效果
- 將原始遙測資料重塑為 AI 就緒輸入內容
- 套用採用 Google Gemini 技術的策略層
- 結合模型輸出內容、編碼的人工指引和安全規則
- 透過使用者介面提供教練回饋

學習目標

完成本程式碼研究室後，您將能夠：

- 說明如何讓 AI 系統更值得信賴
 - 說明模組化 AI 架構的用途
 - 建立簡單的模擬遙測管道
 - 準備實用的結構化資料，供 LLM 使用
 - 套用防護機制和人工輔助規則，提升信任度
 - 評估如何將這項架構套用至其他網域
-

步驟 2 · 約 0 分鐘

2. 所需條件

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

[info](#)

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

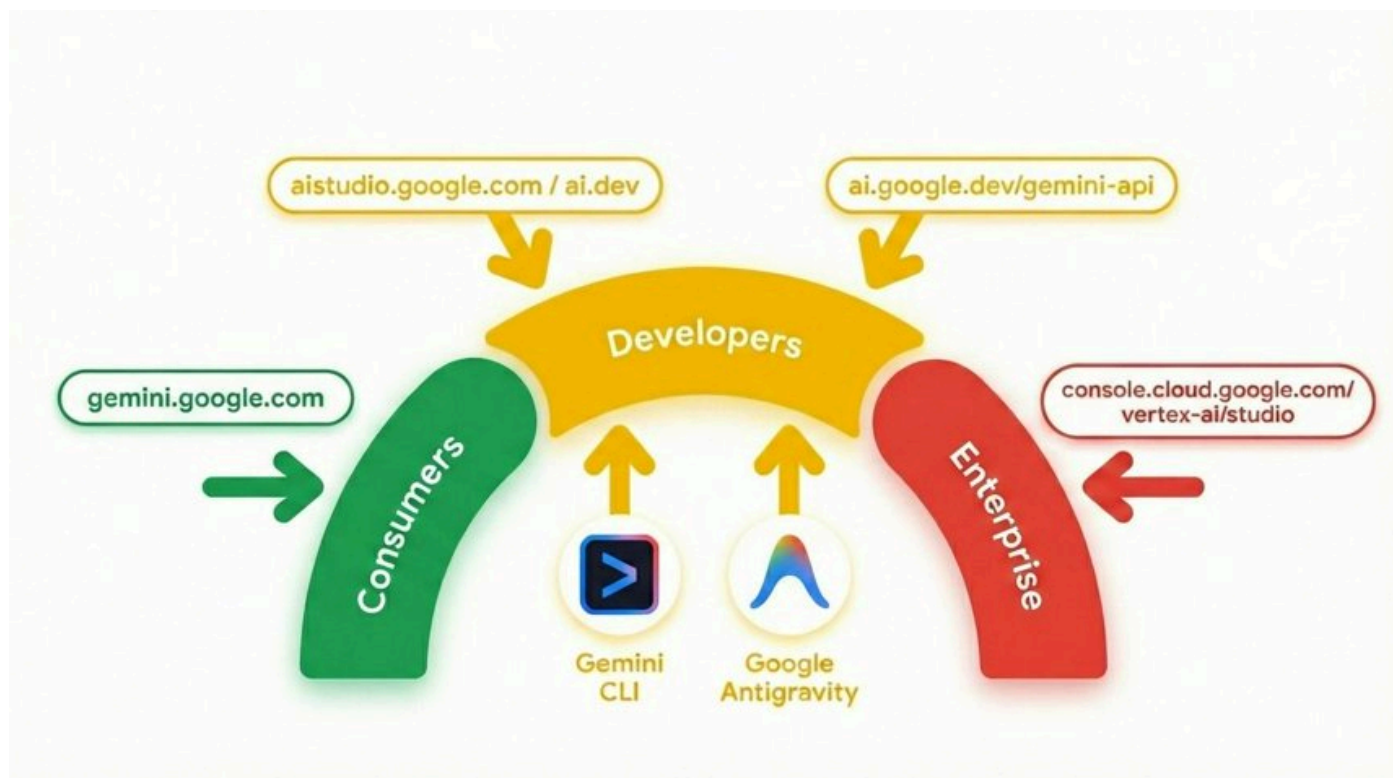
開始前，請確認您已備妥必要的帳戶、工具和服務。

必要條件

您應該：

- 使用 Gmail 地址的個人 Google 帳戶
- 存取 Google Cloud，並對 CLI 有基本瞭解
- 有效的帳單帳戶或雲端抵免額
- 對 Google Cloud 和使用 Gemini 的生成式 AI 有基本瞭解

Gemini 是 Google 的 AI 模型，以最先進的推論技術為基礎，能將任何想法化為現實。這款模型非常適合多模態理解、代理功能和直覺式程式開發。



3. 為什麼可信賴的 AI 很重要

許多 AI 系統都能生成流暢且令人信服的回覆，但流暢並不等於可信。在實際系統中，使用者通常需要及時、有根據的回覆，且這些回覆會受到安全規則限制，並由領域專業知識塑造。

如果系統處理的資料變動快速，這點就特別重要。如果回覆太慢，可能就沒有用處。如果回覆聽起來很有自信，但忽略重要背景資訊，就可能誤導使用者。即使回覆內容聽起來很專業，但如果與人類專業知識無關，可能就難以信任。

在本程式碼實驗室使用的賽車情境中，問題並非 AI 能否說出有趣的話，問題在於系統能否提供實用、安全、及時且適合當下情況的建議。

讓我們看看一小段遙測資料範例，並比較兩種可能的輸出內容：

```
Racing Car Telemetry Data
{
  "speedMph": 118,
  "throttle": 91,
  "frontGrip": "nominal",
  "rearGrip": "low",
  "trackPosition": "Turn 1 Entry"
}
```

未經查證的 AI 回覆

```
"Stay aggressive on the throttle and carry your speed into Turn 1"
```

信任感回覆

```
"Rear grip is low at Turn 1 entry. Reduce your throttle slightly and prioritize a stable corner entry"
```

看到兩者的差異了嗎？

如果我們只依賴未經訓練的 AI 回覆，會發生什麼事？

第一個回覆聽起來很有把握，但忽略了風險。第二個回覆反映了背景資訊和限制，因此更實用。

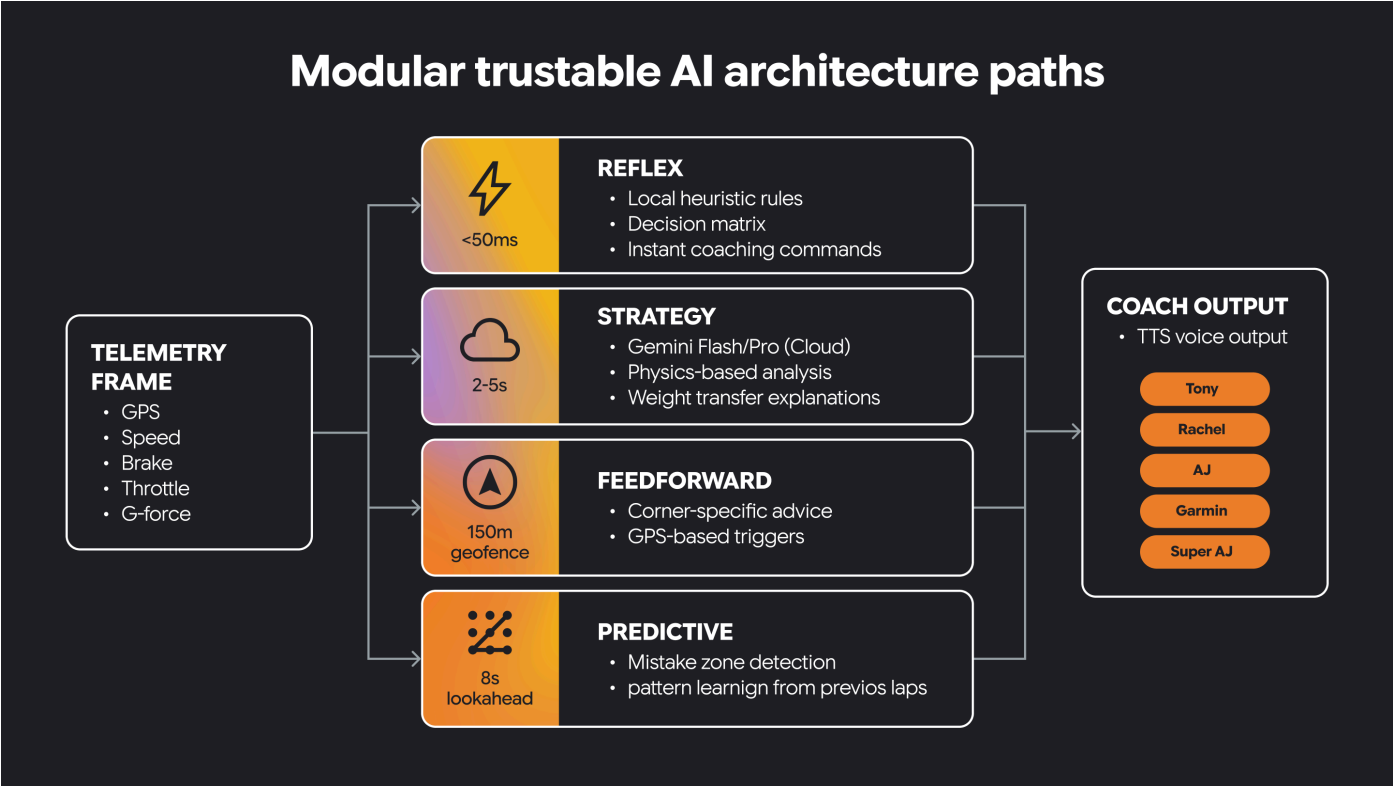
您應將 LLM 視為廣泛架構的一部分，而非整個系統，以提高可信度。此外，許多應用程式需要快速提供建議，才能採取行動，例如賽車、醫療程序、航空、電網、交易系統、航海導航等。

現在，我們來瞭解如何建立這類架構。

4. 瞭解高速 AI 和模組化可信架構

部分 AI 系統需要非常不同的行為。必須快速因應變化，同時支援較慢但更周全的推理。

模組化架構會將這些責任劃分為不同的路徑。其中一個路徑可以是反射路徑，負責處理傳入信號的即時解讀作業，這類作業對時間要求嚴格。另一條路徑則可著重於策略，支援更高層次的推理，以及更瞭解情境的決策。其他路徑則會指定其他類型的功能。



有些決策必須即時做出。有些決策需要較長時間思考。

可信賴的 AI 通常需要兩者兼具。

這種架構分離方式有助於系統保持回應能力，同時支援更豐富的 AI 輔助指引。此外，這也是導入人為引導限制和領域知識的明確位置。

在這個小型程式中，我們以 Python 函式實作了反射路徑和策略路徑。

```
const telemetry = {
  speed: 147,
  grip: 0.68,
  corner_type: "sharp",
  lap_trend: "entering_corners_too_fast",
};

function reflexPath(event: typeof telemetry): string {
  if (event.grip < 0.70) {
    return "REFLEX: Reduce throttle now";
  }
  return "REFLEX: No urgent issue";
}

function strategyPath(event: typeof telemetry): string {
  if (event.lap_trend === "entering_corners_too_fast") {
    return "STRATEGY: Brake earlier and prioritize corner exit";
  }
  return "STRATEGY: Driving pattern looks stable";
}

console.log(reflexPath(telemetry));
console.log(strategyPath(telemetry));
```

這兩個函式在相同遙測資料下的行為不同。反射功能會立即發出警告。策略功能會根據規則提供教練建議。

您認為將這項邏輯分開有什麼好處？

現在，讓我們建構一個有趣的多部分應用程式，看看這個架構如何將快速回應和深入推論轉化為可信賴的 AI 系統，讓您實際體驗。

步驟 5 · 約 0 分鐘

5. 建構遙測串流伺服器

瞭解架構目標後，即可開始建構驅動應用程式的資料管道。

在本節中，您將為虛擬賽車建立簡單的遙測資料串流。資料會來自含有 GPS 或軌跡位置資料的 CSV 來源，而應用程式會將資料轉換為即時串流，供 UI 和 AI 層使用。

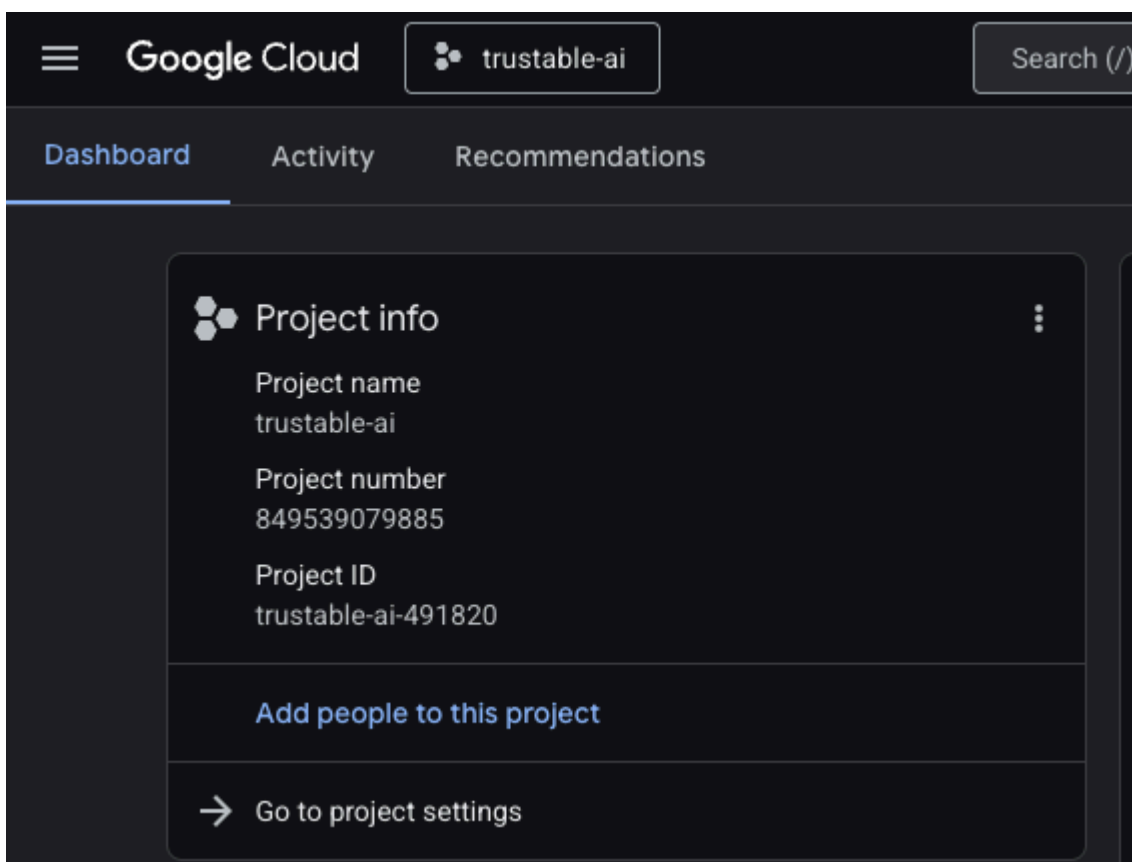
本節內容如下：

- 在 Google Cloud 中為串流伺服器和應用程式建立新專案
- 建立小型伺服器來發出遙測資料
- 將這些事件串流至瀏覽器 UI 或控制台

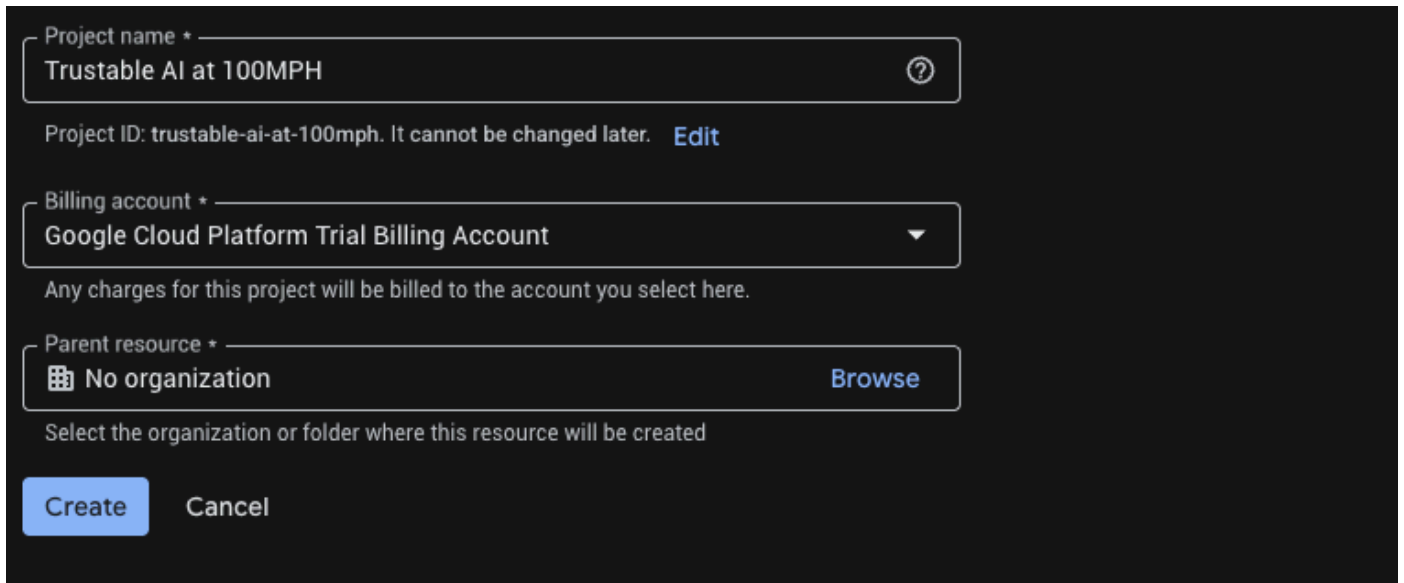
1. 開啟 Cloud Shell

A. 前往 [Google Cloud 控制台](#)。

B. 為這個程式碼研究室建立新專案。按一下頂端的專案下拉式選單。



建立專案時，不妨順便連結帳單帳戶：



(選用) 如果您已建立專案，可以開啟左側面板，按一下 **Billing**，然後檢查帳單帳戶是否已連結至這個 **GCP** 帳戶。

C. 取得 Gemini API 金鑰

啟用 Google Cloud 抵免額後，您需要 Gemini API 金鑰才能在 Google Cloud 中存取 Gemini。

如要建立 Gemini API 金鑰，我們需要使用 [Google Vertex AI Studio](#) 生成金鑰。

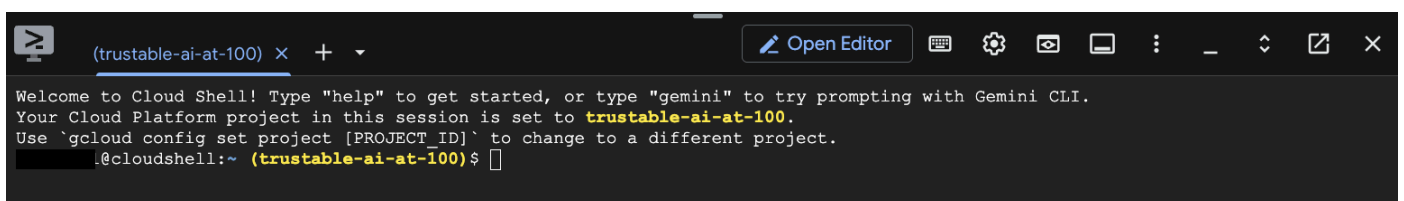
在 Vertex AI Studio 中，按一下左下角「文件」上方的「取得 API 金鑰」。建立 Gemini 的 API 金鑰 (看起來像是一長串隨機字元)。請將這組金鑰妥善儲存於安全的位置。我們會在步驟 6「建構賽車模擬器」中使用這個 API 金鑰，驗證我們對 Google Cloud 中 Gemini 的存取權。

請勿在原始碼中加入 API 金鑰，也不要提交至 Git 存放區。建議將這些資訊儲存在環境變數或安全的密鑰管理工具中。請將 API 金鑰視為密碼，不要在即時通訊、電子郵件、螢幕截圖或記錄檔中分享。定期輪替。對於正式版系統，您應新增監控和用量快訊，以便快速偵測任何濫用行為，並立即撤銷遭盜用的金鑰。

D. 點選頂端列中的「Cloud Shell」圖示 (終端機圖示)，開啟以瀏覽器為基礎的終端機。



E. 等待終端機工作階段啟動。



2. 取得程式碼

複製主要存放區。

```
git clone https://github.com/ocupop/trustable-ai-codelab.git
cd trustable-ai-codelab
```

請注意，這個存放區中有兩個資料夾：「koru-application」（網頁應用程式）和「streaming-telemetry-server」（模擬的即時賽車遙測資料）。這個步驟說明「streaming-telemetry-server」。我們會在下一個步驟中使用「koru-application」。

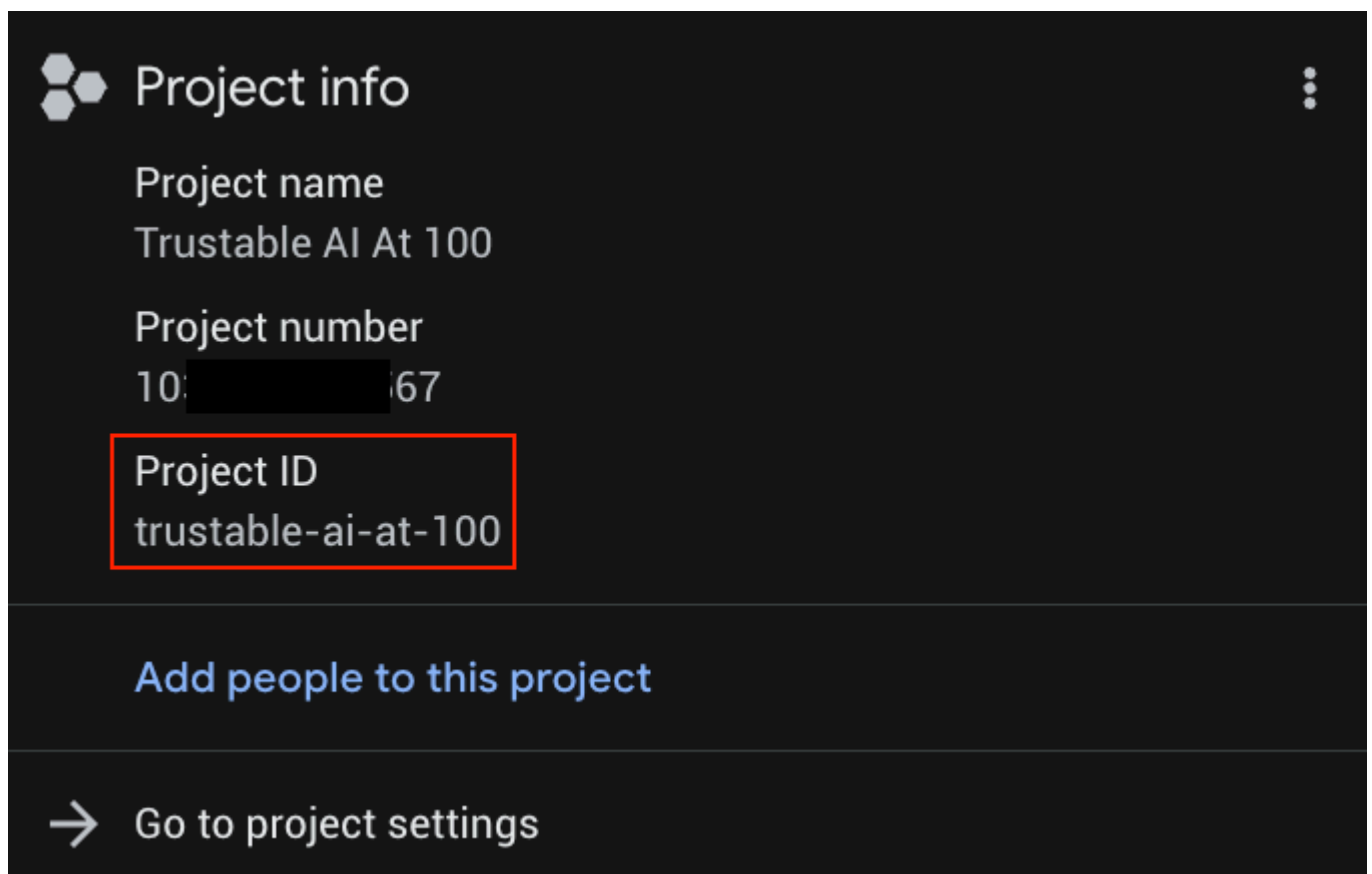
3. 啟用必要的 API

每個專案執行一次：

```
# Set Project ID
gcloud config set project YOUR_PROJECT_ID
# Enable APIs
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com
```

將 `YOUR_PROJECT_ID` 替換為實際專案 ID (如果已設定專案，則略過第一行)。

您可以在專案清單中找到 `YOUR_PROJECT_ID`



4. 將後端部署至 Cloud Run

從存放區根目錄 (即確認您位於 `trustable-ai-codelab` 資料夾中)：

```
gcloud run deploy streaming-telemetry-server \
  --source streaming-telemetry-server \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated
```

請注意，系統可能會提示你按下「Y」

- 首次執行時，系統可能會提示您啟用 API 或建立 Artifact Registry 存放區，請視需要接受。
- 如果您使用的區域不是 `us-central1`，請使用 `--region` 指定該區域
- 部署完成後，gcloud 會列印 **service-URL**。我們只需要在這個網址中附加「events」，即可將其做為遙測伺服器的完整端點。

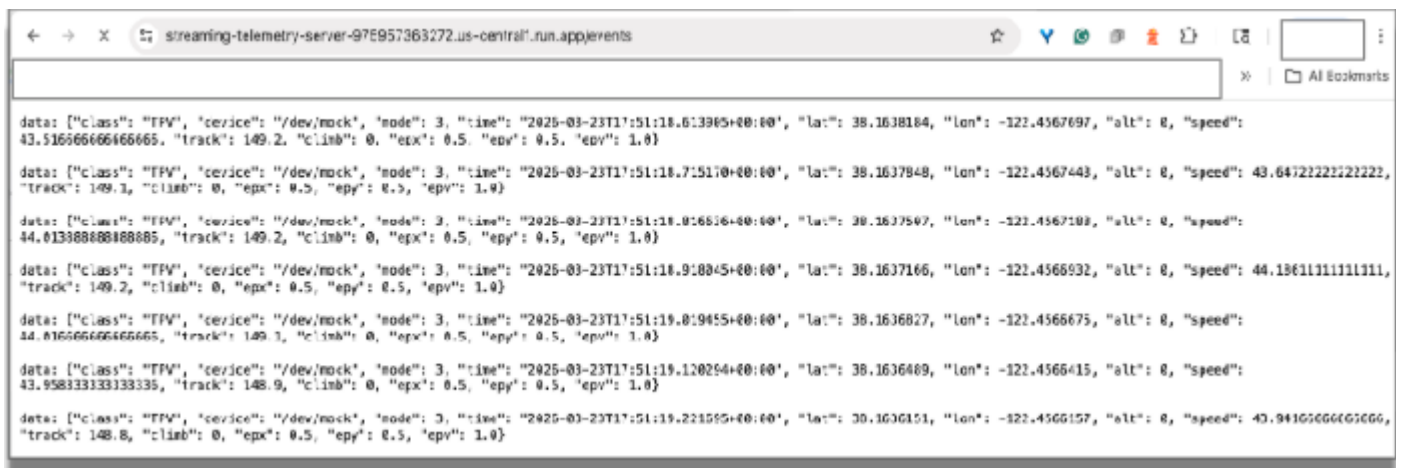
5. 使用串流 URL

遙測伺服器現在會使用伺服器傳送事件 (SSE)，在以下形式的端點發出模擬遙測資料：

```
service-URL/events // service-URL - the last line displayed by "deploy"
```

串流端點的格式為：`https://streaming-telemetry-server-${PROJECT_NUMBER}.${REGION}.run.app/events`，其中 `$PROJECT_NUMBER` 和 `$REGION` 會設為特定伺服器的值。

在瀏覽器中測試：使用 Chrome 造訪這個串流端點網址。您應該會在瀏覽器中看到傳入的串流資料，模擬賽車感應器發出的資料。



關閉瀏覽器分頁即可終止連線。

使用 `curl` 進行測試：

現在，我們來透過 Shell 指令列進行測試。

```
curl -N service-URL/events # Replace service-URL with actual deployment endpoint
```

雲端 Shell 視窗中應會顯示傳入的串流資料。

```
Terminal (trustable-3) X +
service url: https://streaming-telnet-ter-... .run.app
fgreen@trustable-ai-modelab (trustable-ai) $ curl -H "https://streaming-telnet-ter-... .run.app/event"
data: {"class": "FPV", "device": "/dev/mock", "node": 1, "time": "2026-13-23T11:57:43.548968-00:00", "lat": 38.1626113, "lon": -122.461776, "alt": 0, "speed": 21.025, "track": 267.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 1, "time": "2026-13-23T11:57:43.669875-00:00", "lat": 38.1626113, "lon": -122.461709, "alt": 0, "speed": 21.319446666666666, "track": 261.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:43.751821-00:00", "lat": 38.1626113, "lon": -122.4617321, "alt": 0, "speed": 21.95833333333333, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:43.852714-00:00", "lat": 38.1626117, "lon": -122.4617753, "alt": 0, "speed": 21.016666666666666, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:43.950113-00:00", "lat": 38.1626117, "lon": -122.4617084, "alt": 0, "speed": 21.052777777777777, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.050719-00:00", "lat": 38.1626113, "lon": -122.4617316, "alt": 0, "speed": 21.06111111111111, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.156145-00:00", "lat": 38.1626143, "lon": -122.4617746, "alt": 0, "speed": 21.16611111111111, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.257538-00:00", "lat": 38.1626113, "lon": -122.4617076, "alt": 0, "speed": 21.213888888888888, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.359111-00:00", "lat": 38.1626113, "lon": -122.4617303, "alt": 0, "speed": 21.244444444444444, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.460547-00:00", "lat": 38.1626113, "lon": -122.4617729, "alt": 0, "speed": 21.269444444444444, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}

data: {"class": "FPV", "device": "/dev/mock", "node": 3, "time": "2026-13-23T11:57:44.562050-00:00", "lat": 38.1626113, "lon": -122.4617055, "alt": 0, "speed": 21.327777777777777, "track": 271.3, "climb": 0, "apx": 0.5, "apy": 0.5, "apz": 1.0}
```

我們會使用這項遙測資料，模擬賽車感應器發出的資料。本程式碼研究室的其餘部分會使用這項資料。在終端機視窗中輸入 CTRL-C，即可終止 curl 程式。

注意事項

完成這個部分時，請注意傳入資料的性質。原始遙測資料通常量大、時效性高，且不適合立即用於 AI 推理。建構前端應用程式後，我們需要將原始資料篩選為有效率的格式，供 LLM 快速處理。

但首先，請建構網頁前端，將資料視覺化。

步驟 6 · 約 0 分鐘

6. 建構賽車模擬器

本節內容如下：

- 建構賽車模擬
- 將遙測伺服器連線至賽車網頁應用程式
- 查看模擬賽事

此時，我們已在雲端執行賽車的遙測模擬作業。現在，請建構在本機電腦上執行的應用程式，連線至 Google Cloud 並將資料視覺化。

這款值得信賴的 AI 應用程式結合了 Google Cloud 服務的強大功能和彈性，以及在 Chrome 中執行的本機智慧功能。

串流遙測服務在 Google Cloud 中執行，但賽車應用程式在本機執行。也就是說，您必須再次複製存放區，這次是複製到筆電或桌上型電腦。

為簡化作業，串流伺服器和賽車應用程式的程式碼都位於同一個存放區。

從 GitHub 複製前端應用程式：

```
git clone https://github.com/ocupop/trustable-ai-codelab.git
cd trustable-ai-codelab
```

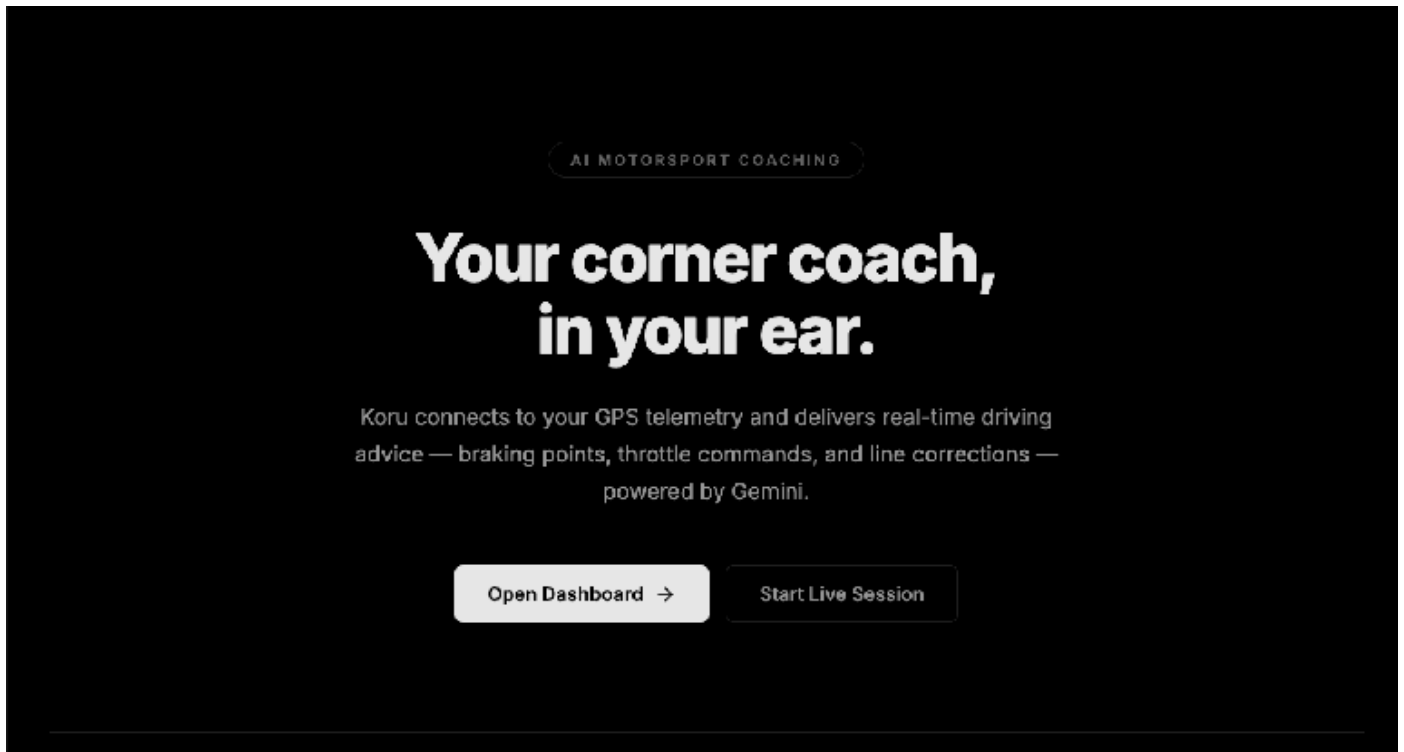
在筆電或桌機上複製存放區後，我們來執行應用程式。

```
cd koru-application          # racing car simulation app
npm install
npm run dev
```

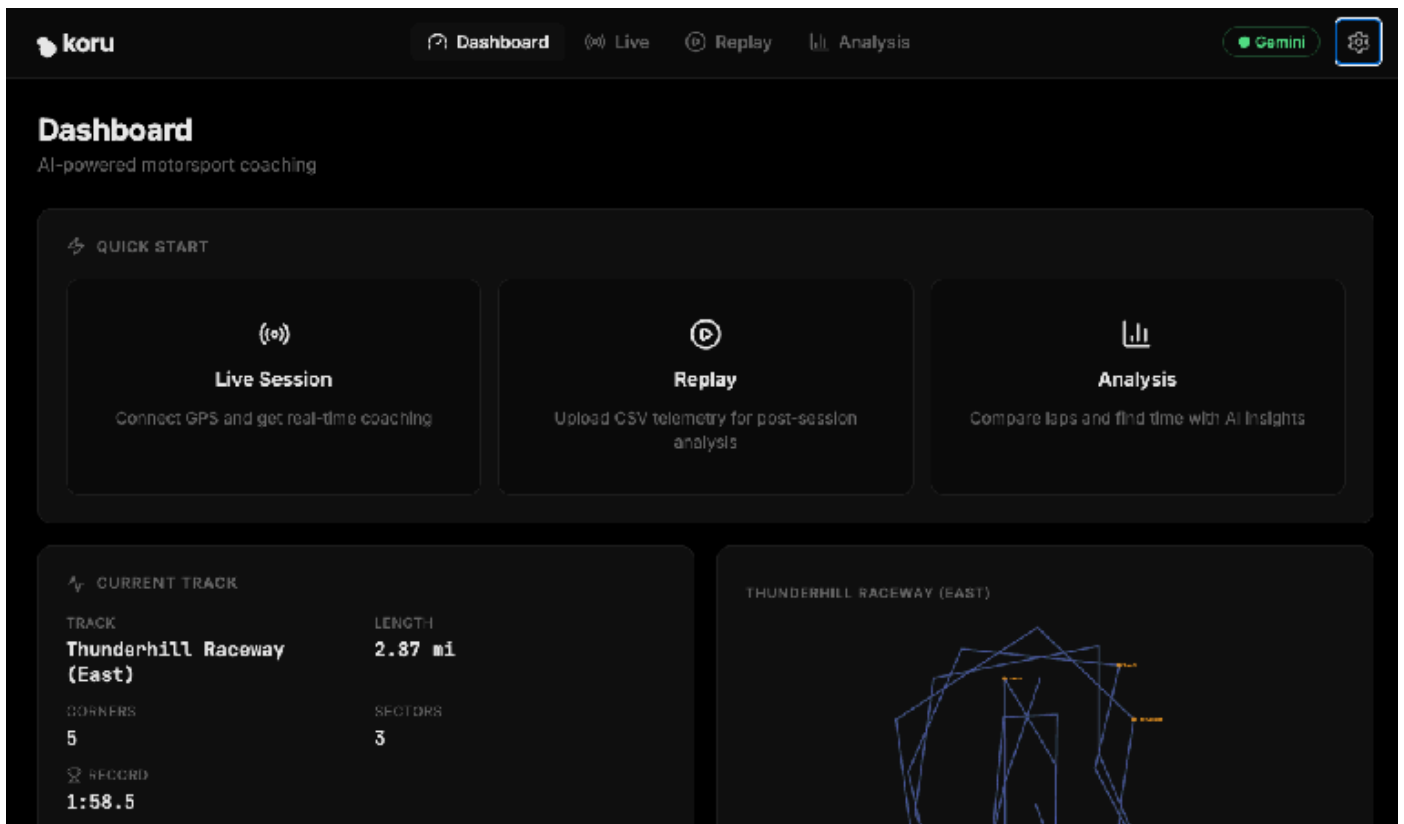
「koru」是紐西蘭毛利文化的符號，代表新的開始。

```
VITE v8.0.0 ready in 114 ms
+ Local:   http://localhost:5173/
+ Network: use --host to expose
+ press h + enter to show help
```

在 Chrome 中開啟本機電腦上的通訊埠 (如上例中的 <http://localhost:5173>)。系統會顯示「AI Motorsport Coaching」應用程式的到達網頁。

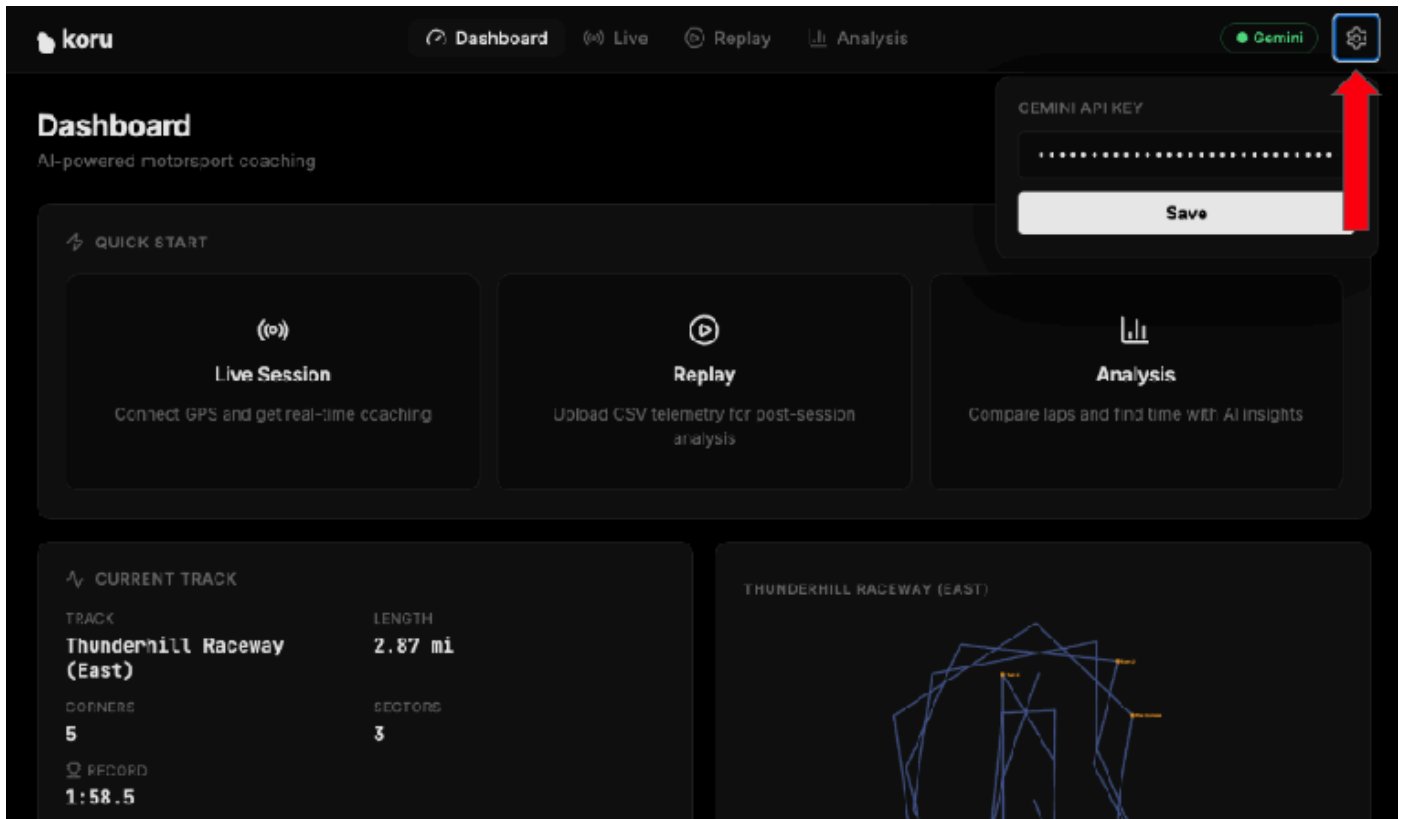


按一下「Open Dashboard ->」按鈕。這會啟動應用程式的 UI。



到目前為止，您已在 Google Cloud 中建立遙測伺服器，產生模擬賽車遙測資料，並建立可將資料視覺化及連線至 LLM 的本機網頁應用程式。讓我們連結這些服務，以及 Gemini LLM 服務。

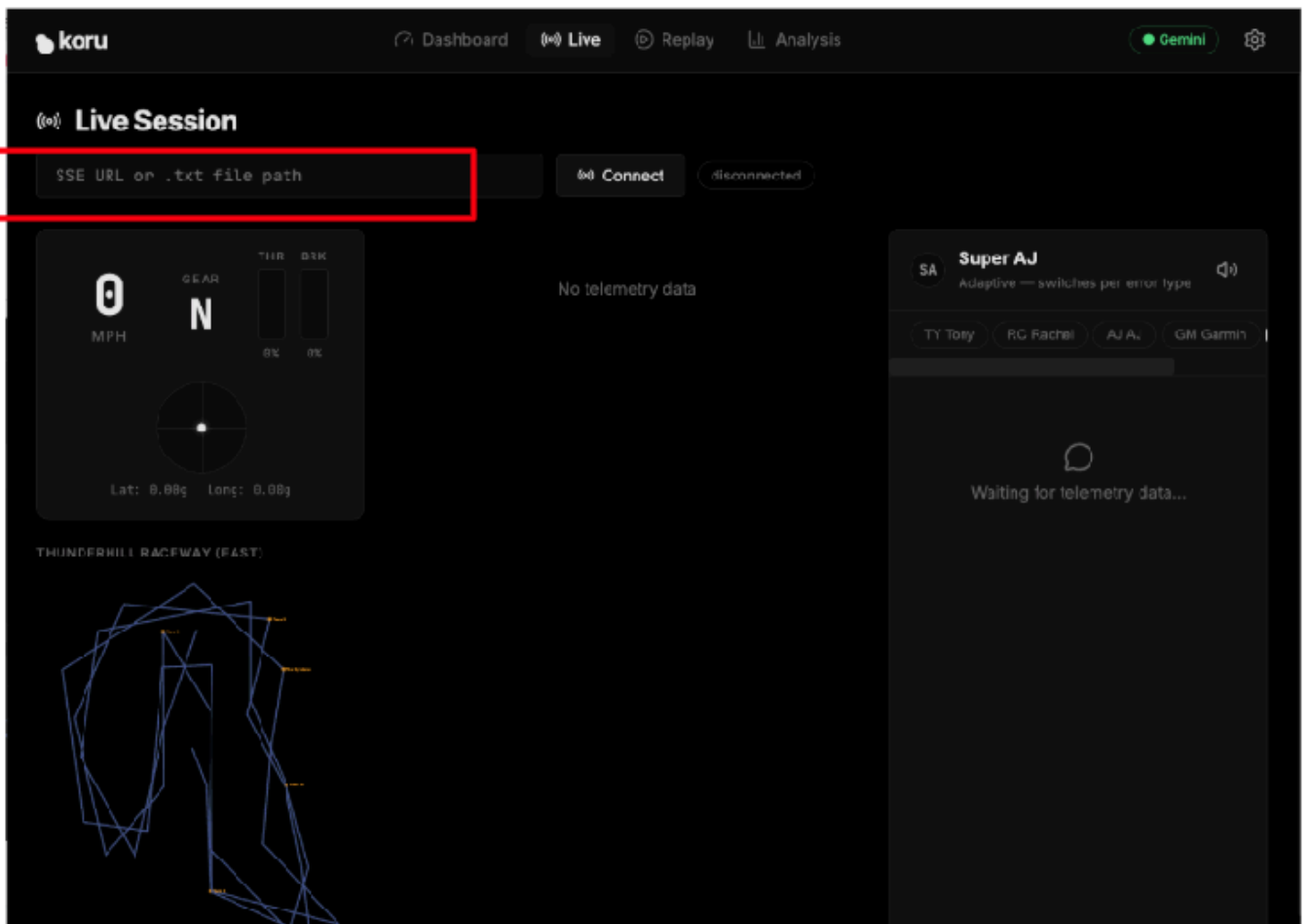
按一下應用程式右上角的齒輪圖示 (設定)。



輸入步驟 2 中的 Gemini API 金鑰。這樣一來，您就能在 Google Cloud 中存取 Gemini 服務。

按一下「儲存」，讓應用程式記住您的 API 金鑰。

現在，請將應用程式連線至遙測伺服器。在應用程式資訊主頁中，按一下「Live Session」。



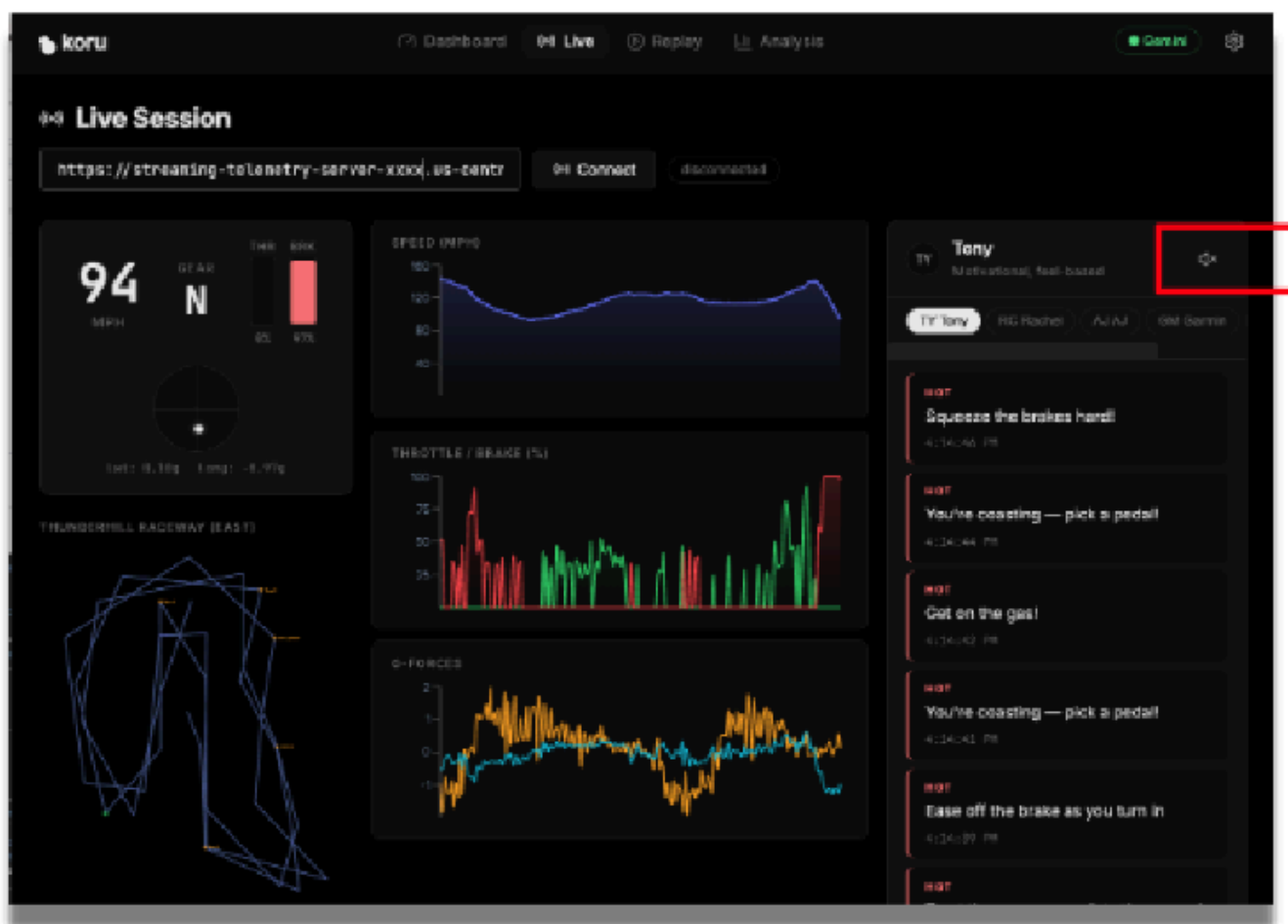
在「SSE URL or .txt file path」文字欄位中，輸入雲端遙測伺服器的特定網址 (步驟 5)。我們的 SSE 網址格式如下：

```
https://streaming-telemetry-server-${PROJECT_NUMBER}.${REGION}.run.app/events
```

輸入遙測伺服器端點網址後，按一下文字欄位右側的「連線」。請記得在網址結尾加上「events」。

現在您應該會看到應用程式以視覺化方式呈現模擬資料！

調高音箱音量後，你就能聽到不同類型教練提供的賽車建議。每位教練的個性都不一樣，你可以嘗試選取不同教練，聽取他們提供的各種賽事建議和不同風格的語音。如有需要，可以按一下喇叭圖示停用音訊。



現在我們已有可運作的應用程式，接下來要探討如何準備資料，讓 LLM 能夠有效處理，以及如何新增其他功能，提升整體系統的可信度。

7. 準備 AI 推理的遙測資料

原始遙測資料有助於模擬，但通常過於詳細且傳送頻率過高，不適合直接傳送至 LLM。如果傳送的遙測資料完全未經修改，可能會增加延遲時間、產生雜訊，並降低最終指引的品質。

在本節中，您將重新調整遙測資料的形狀，使其更有用。

本節內容如下：

- 檢查原始遙測資料 JSON
- 找出與推論最相關的欄位
- 篩選或摘要資料
- 減少不必要的細節
- 準備 AI 友善的駕駛狀態表示法

這是建立可信賴 AI 的重要步驟。回覆內容的品質不僅取決於模型，也取決於模型接收的資料結構和相關性。

現在來看看賽車的具體資料。我們可以變更應用程式中的特定值、重新載入，並觀察結果，藉此進行實驗。

```
../src/services/telemetryStreamService.ts near line 180
```

```
// Clamp G-forces
gLat = Math.max(-3, Math.min(3, gLat));           // sideways G-force
gLong = Math.max(-3, Math.min(3, gLong));          // front/back G-force
```

車輛的 G 力可測量加速或減速。在賽車中，瞭解重力有助於掌握車輛操控和整體性能。如果應用程式沒有這項資訊，就難以提供建議給駕駛人。將這兩行註解掉，將 `gLat` 和 `gLong` 值都設為 0.0，然後重新執行應用程式。

請注意，車輛接近彎道時，系統不會提供任何建議。這對賽車手來說沒什麼幫助！

然後還原變更並重新執行應用程式。請注意，車輛抵達轉角時，系統會提供實用的語音建議。系統會根據 G 力資料點提供駕駛建議。

現在，我們將車輛速度人為限制在 30 英里/時的悠閒步調。以這種速度我們無法贏得任何賽事，但肯定能展現我們接受的教練指導類型。

在同一個檔案 ([telemetryStreamService.ts](#)) 的第 158 行附近，您會找到 `processPoint()` 函式。讓我們在該函式中限制速度。

變更：

```
private processPoint(point: GpsSSEPoint) {
  ...
  const speedKmh = point.speed > 200 ? point.speed : point.speed * 3.6;
  ...
}
```

收件者：

```
private processPoint(point: GpsSSEPoint) {  
  ...  
  let speedKmh = point.speed > 200 ? point.speed : point.speed * 3.6;  
  speedKmh = Math.min(speedKmh, 48);    // 48 kmh is approx 30 mph  
  ...  
}
```

重新執行應用程式。現在我們會收到哪種教練建議？如果只是悠閒地開車，就不需要太多。

現在請還原這些變更，然後重新執行應用程式。

顯然，車速是很有價值的資料點。請務必瞭解哪些特定資料對於提供實用建議至關重要。評估哪些資料不相關也同樣重要。

您也應該開始思考安全和信任問題。即使輸入內容經過充分準備，也無法保證生成可靠的答案。我們仍需導入人為指引的規則和明確的限制。

資料準備不只是預先處理步驟，這是信任策略的重要環節。輸入內容越乾淨，通常就能獲得更精確可靠的輸出內容。

8. 新增防護措施和編碼的人類專業知識

可信賴的 AI 系統不應只依賴模型輸出內容，在許多情況下，最可靠的系統會結合大型語言模型推論、明確規則、領域知識和人為引導的限制條件。

在本節中，您將新增該圖層。

您可以將這個層視為編碼的教練知識。這類資訊可能包括偏好的回覆模式、驗證規則、安全檢查或結構化指引，有助於系統保持實用性。

本節內容如下：

- 導入可影響模型行為的回覆規則
- 進行安全檢查，減少誤導性建議
- 將編碼的人類專業知識納入管道
- 比較加入這些內容前後的回覆

現在來瞭解如何將網域專業知識新增至應用程式。

LLM 通常不會接受賽車或賽車性能物理學方面的訓練。如果應用程式包含該領域的專業知識，使用者就會更信任其指引。這些指引來自以人類專業知識為基礎的規則，也就是**領域專業知識層**。

```
../src/utils/coachingKnowledge.ts near line 115
```

```
...
export const RACING_PHYSICS_KNOWLEDGE = `
CORE PRINCIPLES:
1. **The Friction Circle:** A tire has 100% grip. If you use 100% for braking, you have 0%
for turning.
  - *Error:* Turning while 100% braking = Understeer (Plowing).
  - *Fix:* "Trail braking" (releasing brake pressure as steering angle increases).

2. **Weight Transfer:**
  - Braking shifts weight forward (Front grip UP, Rear grip DOWN).
  - Accelerating shifts weight backward (Front grip DOWN, Rear grip UP).
  - *Error:* Lifting off throttle mid-corner shifts weight forward abruptly -> Oversteer
(Spin risk).

3. **The racing line:**
...
`
```

這些賽車原則是提供可信賴輸出內容的關鍵要素。如果我們沒有這項專業知識，會發生什麼事？讓我們一起來看看。

讓我們移除 `RACING_PHYSICS_KNOWLEDGE`，並探索賽車建議。

```
export const RACING_PHYSICS_KNOWLEDGE = ``;
```

重新執行應用程式。現在會收到哪種教練建議？

請注意一般建議。

我們無法再取得摩擦力、重量轉移、出口速度等詳細資訊，因此可信度降低。請還原該賽車專業知識，然後重新執行應用程式。

這是可信賴 AI 系統的重要環節。更強大的提示不會神奇地建立信任感。信任感來自系統設計和批判性思考。

LLM 是解決方案的一部分，但不是整個解決方案。如果 AI 輸出內容是以明確的人類知識為依據，可信度就會提高。

9. 設計教練角色和使用者的體驗

推論管道就位後，下一個問題是系統應如何與使用者溝通。

在本節中，您將定義策略層與駕駛人的溝通方式，塑造教練體驗。您將為其中一個教練角色調整系統提示，並考量如何清楚、及時且最重要的是可據以行動地提供指引。

本節內容如下：

- 建立或修正教練角色的系統提示
- 嘗試不同的教練風格
- 觀察提示變更對回覆的影響
- 定義可信回饋的 UI 需求
- 瞭解文字轉語音 (TTS) 功能對緊急和非緊急訊息的支援

我們的應用程式包含多個教練角色。每種建議提供的教練指導類型都不相同。

PERSONA	特性
Tony	激勵人心、以感受為基礎
Rachel	技術性強，著重物理原理
AJ	直接、生硬的指令
Garmin	以資料為中心，進行差異最佳化
Super AJ	自動調整，每種錯誤類型各一個

這些角色定義在 `../src/utls/coachingKnowledge.ts` 檔案中。

在這個檔案中，您會看到將字串鍵與 `CoachPersonas` 建立關聯的物件對應 (`COACHES`)。 `CoachPersona` 包含各類教練的屬性。其中一項重要屬性是 `systemPrompt`。每個角色都有自己的 `systemPrompt`，可引導 LLM 如何回覆。

我們來修改其中一個 `system prompts`，看看 LLM 會如何回應。

在第 31 行附近，您會看到「AJ」的 `systemPrompt`，他的建議非常直接且坦率。我們來變更 `systemPrompt`，讓 AJ 變得過於客氣。

```
systemPrompt: `You are AJ, a race engineer that is excessively polite.
  Use telemetry terminology. Be actionable
  Examples:      "Lat G settling. please throttle",
                  "Brake when its convenient."
  Keep responses under 12 words. Never explain — just command.`
```

重新執行應用程式，選取 AJ 做為教練，然後查看系統生成的回覆類型。

現在請還原原始 `systemPrompt`，然後重新執行應用程式。請注意，系統提示詞對於引導 LLM 提供符合角色設定的回覆至關重要。

信任感不僅來自正確性，也與交付有關。即使建議在技術上正確無誤，如果內容不清楚、時機不當或令人分心，仍可能無效。

值得信賴的系統必須能妥善溝通。使用者體驗是信任架構的一部分。

在正式環境中，您通常會為每位教練指派不同的聲音，而不是使用單一聲音並改變音調和語調。不過，使用文字轉語音服務會大幅增加網路傳輸的權杖數量。如果使用正式版雲端帳戶，就能聽到不同類型的教練語音。

步驟 10 · 約 0 分鐘

10. 查看端對端架構

到目前為止，您已建構系統的主要部分。現在請退後一步，回顧這兩項功能如何搭配運作。

您的應用程式現在包含下列元件：

- 遙測串流
- 視覺化層
- 支援 AI 的資料轉換階段
- 由推論型 LLM 驅動的策略元件
- 防護機制和編碼的人工指引
- 提供給使用者的指導體驗

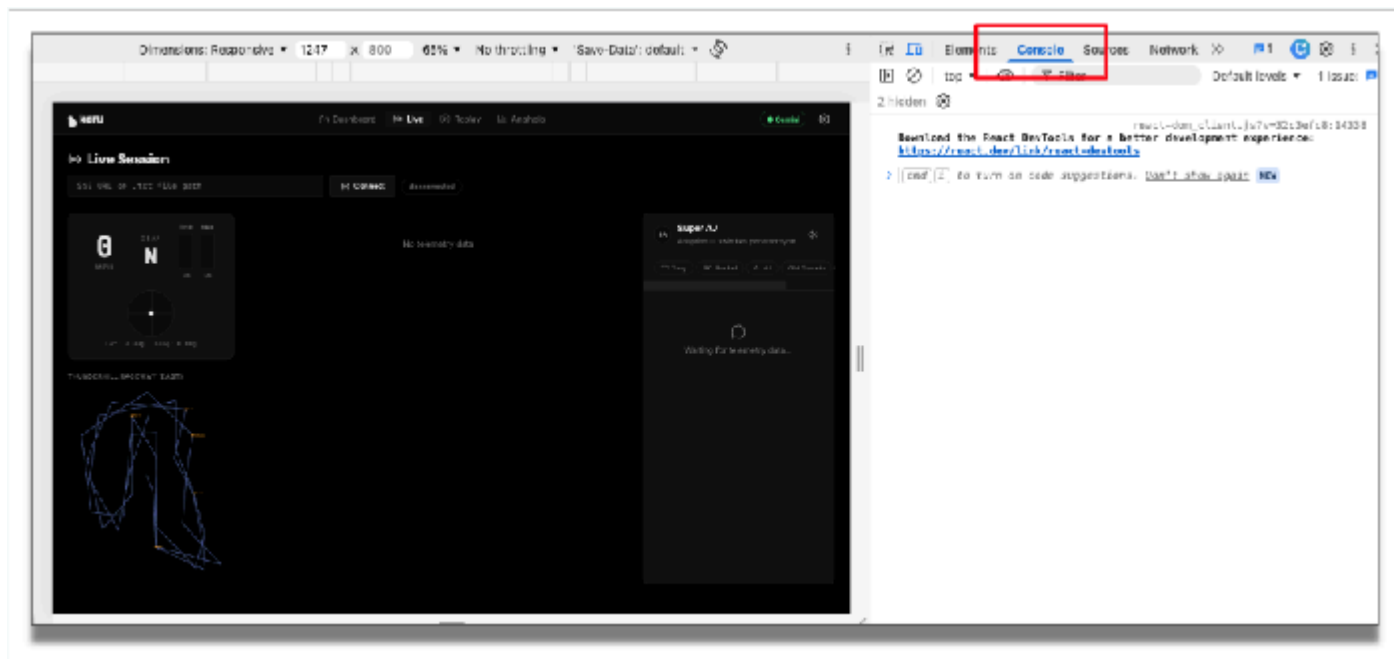
若要瞭解這些元件的整體流程，最簡單實用的方法就是在應用程式中新增記錄。

我們會新增記錄，以便查看遙測資料在路徑中的流動情形。

首先，我們只查看遙測資料。在 `telemetryStreamService.ts` 中，於第 212 行附近 (`this.emit(frame)` 之前)，新增一行程式碼，顯示速度、側向 G 力 (側向加速度) 和駕駛人踩煞車踏板的力道。

```
console.log('FRAME', {
  speed: frame.speed.toFixed(1),
  gLat: frame.gLat.toFixed(2),
  brake: frame.brake.toFixed(0) }
);
```

重新載入應用程式。執行應用程式前，請先在 Chrome 開發人員工具中開啟控制台，查看這項偵錯資訊。



在應用程式中輸入遙測端點，然後按一下「連線」。現在您應該會看到傳入的遙測資料。

現在，讓我們為反射路徑和策略路徑新增記錄。

在 `../src/services/coachingService.ts` 的 `this.emit()` 前方第 71 行附近，為 **reflex** 路徑新增記錄行：

```
console.log('Reflex', {
  action: rule.action,
  text,
  coach: this.coachId }
);
```

在同一個檔案中，大約在第 287 行，於 `this.emit()` 前方新增 **strategy** 路徑的類似記錄行 (我們來新增 Gemini API 傳回的輔導回覆 `text`)：

```
console.log('Strategy', {
  coach: coach.id,
  chars: text.length,
  preview: text.slice(0, 60) }
);
```

重新執行應用程式。您會在控制台中看到遙測資料如何從來源流經這些路徑。系統會篩選傳入的串流、傳送至 LLM、透過值得信賴的專家驗證，然後透過適當的使用者介面呈現給使用者。

請注意，我們已連結各種技術元件，以達成可信賴 AI 的宏大目標。架構的價值並非來自單一元件，價值來自於各個部分如何相互強化。

可信賴的 AI 是架構成果，而非單一功能。

拆除 (移除服務)

請記得在不再需要服務時移除。測試完遙測伺服器 and 應用程式後，請刪除 Cloud Run 服務，停止計費：

```
gcloud run services delete streaming-telemetry-server \
  --region us-central1 \
  --platform managed
```

請記得視需要將 `us-central1` 替換為部署時使用的區域。當系統提示時，確認刪除。

選用：容器映像檔、套件和相關構件會儲存在 Artifact Registry 中。為避免產生舊有未使用資料的儲存費用，請移除這些資料，以免產生不必要的費用。如要從 Artifact Registry 移除資料，請使用 Cloud Console：Artifact Registry，然後選取部署作業建立的存放區，並刪除映像檔或存放區。

11. 挑戰

現在核心應用程式已可運作，您也瞭解各種元件，請嘗試擴充設計。

建議的挑戰

- 將更多指導邏輯移至邊緣
- 修改模擬設定，支援下雨或降低牽引力
- 瞭解模型調整或[微調](#)如何提升成效
- 為其他領域 (例如醫療、製造或物流) 調整架構

舉例來說，將本實驗室學到的內容套用至其他網域時，請考慮下列問題：

- 在其他領域中，相當於賽車遙測 (即連續資料) 的概念是什麼？
- 哪些決策需要立即做出，哪些決策更具策略性？
- 需要編碼哪種人類專業知識？
- 使用者需要看到什麼，才會相信系統值得信賴？

這些挑戰可鼓勵您思考賽車範例以外的內容，並瞭解本程式碼研究室背後更廣泛的信賴度設計模式。

12. 總結與後續步驟

在本程式碼研究室中，您建構的不只是賽車示範應用程式，您已建構具體範例，瞭解如何設計值得信賴的 AI 系統。

您從原始遙測資料開始，將資料轉換為 LLM 適用的格式、套用 AI 推理，並透過編碼的人工指引和回應限制，強化輸出內容。您也瞭解到，信任感來自架構，而不只是模型輸出內容。

可信賴的 AI 系統通常會結合下列要素：

- 結構化即時資料
- 以模型為基礎的推理
- 編碼網域專業知識
- 明確的防護機制
- 周到的使用者體驗設計

賽車情境有助於將這些想法具體化，但只要 AI 建議必須及時、實用且可靠，就能採用相同方法。